

Lab 6

METHOD

For this lab I used the algorithm for the FFT that was given in class. Then I was able to use the hint given in the lab. Hint: $\text{ifft}(X) = \text{conj}(\text{fft}(\text{conj}(X))) / N$. This technique allowed me to leverage my existing recursive FFT solution from part A in part B. Then for peak picking I was able to use some pretty simple list slicing to return the desired peaks. For denoising it was as simple as cutting out any frequencies below the desired threshold.

APPROACH

The program uses the Cooley-Tukey radix-2 FFT to transform the input data from the time domain into the frequency domain, which allows for easier note detection and denoising. The IFFT uses the principle that $\text{ifft}(X) = \text{conj}(\text{fft}(\text{conj}(X))) / N$. Note detection takes the FFT of the segment, removes negative frequencies, ignores DC term, and then returns the top peaks. Finally the denoising does FFT, filters any coefficients above the threshold zeroing everything else, then inverts the segment back to time domain using IFFT and returns the real (non-complex) part.

REASONING

This all works efficiently by taking advantage of the divide-and-conquer approach to the FFT through Cooley-Tukey. Cooley-Tukey allows for the calculation of a DFT in $O(N \lg N)$ time as opposed to the $O(N^2)$ time of calculating the DFT normally. This is done by splitting the odd and even terms and solving them recursively. Then finally, the results are combined using twiddle factors to adjust phase of the odd indexed terms.

REFLECTION

I used the included tests.py file to ensure that each part of the program did what it was supposed to. Additionally I initially wrote a small script just to test the FFT because there were a few issues getting test to pass at first. The problem came from using a positive exponent in the twiddle factor when common implementations prefer a negative.